

Week 11- Lab 10: Pandas (Questions)

Assignment Submission Guidelines

Steps to Submit Your Lab Assignments Using Google Colab

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM tonight**.
- Late submissions will not be accepted unless prior arrangements have been made by the TAs.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Please name your files as follows: `Lastname_Firstname_AssignmentName`
- Example: `Alex_John_Lab10.ipynb` , and `Alex_John_Lab10.pdf`

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, please contact the TAs immediately **before the deadline** to avoid penalties.

Questions

Use this dataframe to answer the questions:

```
In [86]: import numpy as np
import pandas as pd

data = {
    'name': ['John', 'Jane', 'Mike', 'Anna', 'Lucy', 'Emma', 'Dave', 'Sophia', 'Zhang', 'Liam'],
    'age': [25, 20, 40, 105, np.nan, 30, 45, 38, 50, 27],
    'experience': [2, 3, 5, 8, np.nan, 5, 20, 10, 25, 4],
    'education level': ['Bachelor', 'Master', 'PhD', 'High School', 'PhD', 'Master', 'PhD', 'Bac',
    'salary': ['50000', np.nan, 60000, 65000, 75000, 55000, 120000, 65000, 130000, 50000],
    'location': ['New York', 'San Francisco', 'Austin', 'Beijing', 'Berlin', 'Toronto', 'London',
    'department': ['Marketing', 'Engineering', 'Sales', 'Research', 'Design', 'HR', 'Finance',
    'languages_known': [2, 3, 1, 2, 3, 1, 2, 3, 2, 2]
}

employees_df = pd.DataFrame(data)
```

Question 1: (5 Points)

Display the first three rows of the `employees_df` dataframe.

```
In [72]: # Grab the first three.
first_three = employees_df.head(3)

# There it is.
print(first_three)
```

	name	age	experience	education	level	salary	location	department	\
0	John	25.0	2.0	Bachelor		50000	New York	Marketing	
1	Jane	20.0	3.0	Master		NaN	San Francisco	Engineering	
2	Mike	40.0	5.0	PhD		60000	Austin	Sales	

	languages_known
0	2
1	3
2	1

Question 2: (5 Points)

Identify any missing values in the dataframe.

```
In [73]: # Find the missing stuff.
missing_values = employees_df.isnull()

# Show it.
print(missing_values)
```

	name	age	experience	education	level	salary	location	department	\
0	False	False	False		False	False	False	False	
1	False	False	False		False	True	False	False	
2	False	False	False		False	False	False	False	
3	False	False	False		False	False	False	False	
4	False	True	True		False	False	False	False	
5	False	False	False		False	False	False	False	
6	False	False	False		False	False	False	False	
7	False	False	False		False	False	False	False	
8	False	False	False		False	False	False	False	
9	False	False	False		False	False	False	False	

	languages_known
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False
8	False
9	False

Question 3: (5 Points)

Replace any missing values in the `experience` column with the median of that column.

```
In [74]: median_experience = employees_df["experience"].median()
employees_df["experience"] = employees_df["experience"].fillna(median_experience)

print(employees_df)
```

	name	age	experience	education	level	salary	location	\
0	John	25.0	2.0	Bachelor		50000	New York	
1	Jane	20.0	3.0	Master		NaN	San Francisco	
2	Mike	40.0	5.0	PhD		60000	Austin	
3	Anna	105.0	8.0	High School		65000	Beijing	
4	Lucy	NaN	5.0	PhD		75000	Berlin	
5	Emma	30.0	5.0	Master		55000	Toronto	
6	Dave	45.0	20.0	PhD		120000	London	
7	Sophia	38.0	10.0	Bachelor		65000	Sydney	
8	Zhang	50.0	25.0	PhD		130000	Tokyo	
9	Liam	27.0	4.0	Master		50000	Dublin	

	department	languages_known
0	Marketing	2
1	Engineering	3
2	Sales	1
3	Research	2
4	Design	3
5	HR	1
6	Finance	2
7	IT	3
8	Operations	2
9	Logistics	2

Question 4: (5 Points)

Convert the `salary` column to a numeric data type, handling any non-numeric values by replacing them with the mean salary.

```
In [75]: # Turn salary into numbers.
employees_df["salary"] = pd.to_numeric(employees_df["salary"], errors='coerce')

mean_salary = employees_df["salary"].mean()

# Fill the odd spots.
employees_df["salary"] = employees_df["salary"].fillna(mean_salary)
print(employees_df)
```

	name	age	experience	education_level	salary	location	\
0	John	25.0	2.0	Bachelor	50000.000000	New York	
1	Jane	20.0	3.0	Master	74444.444444	San Francisco	
2	Mike	40.0	5.0	PhD	60000.000000	Austin	
3	Anna	105.0	8.0	High School	65000.000000	Beijing	
4	Lucy	NaN	5.0	PhD	75000.000000	Berlin	
5	Emma	30.0	5.0	Master	55000.000000	Toronto	
6	Dave	45.0	20.0	PhD	120000.000000	London	
7	Sophia	38.0	10.0	Bachelor	65000.000000	Sydney	
8	Zhang	50.0	25.0	PhD	130000.000000	Tokyo	
9	Liam	27.0	4.0	Master	50000.000000	Dublin	

	department	languages_known
0	Marketing	2
1	Engineering	3
2	Sales	1
3	Research	2
4	Design	3
5	HR	1
6	Finance	2
7	IT	3
8	Operations	2
9	Logistics	2

Question 5: (5 Points)

Rename the column `education_level` to `Education_level`.

```
In [76]: # New column name.
employees_df = employees_df.rename(columns={"education_level": "Education_level"})

print(employees_df.columns)
```

```
Index(['name', 'age', 'experience', 'Education_level', 'salary', 'location',
      'department', 'languages_known'],
      dtype='str')
```

Question 6: (5 Points)

Find the average age of employees in the dataframe.

```
In [77]: # Average age.
average_age = employees_df["age"].mean()

print(average_age)
```

```
42.22222222222222
```

Question 7: (10 Points)

Filter the dataframe to display only those employees who have a Master's degree or higher in the `education_level` column.

```
In [78]: # Master's or better.
higher_ed = employees_df[
    employees_df["Education_level"].isin(["Master", "PhD"])
]
```

```
print(higher_ed)
```

	name	age	experience	Education_level	salary	location	\
1	Jane	20.0	3.0	Master	74444.444444	San Francisco	
2	Mike	40.0	5.0	PhD	60000.000000	Austin	
4	Lucy	NaN	5.0	PhD	75000.000000	Berlin	
5	Emma	30.0	5.0	Master	55000.000000	Toronto	
6	Dave	45.0	20.0	PhD	120000.000000	London	
8	Zhang	50.0	25.0	PhD	130000.000000	Tokyo	
9	Liam	27.0	4.0	Master	50000.000000	Dublin	

	department	languages_known
1	Engineering	3
2	Sales	1
4	Design	3
5	HR	1
6	Finance	2
8	Operations	2
9	Logistics	2

Question 8: (10 Points)

Using the `apply` function, create a new column `experience_category` that categorizes employees as "Junior" if they have less than 3 years of experience, "Mid-level" if they have 3 to 7 years, and "Senior" if they have more than 7 years.

```
In [79]: # function for the labels.
def experience_label(x):
    if x < 3:
        return "Junior"
    elif x <= 7:
        return "Mid-level"
    else:
        return "Senior"

# New column.
employees_df["experience_category"] = employees_df["experience"].apply(experience_label)

print(employees_df)
```

	name	age	experience	Education_level	salary	location	\
0	John	25.0	2.0	Bachelor	50000.000000	New York	
1	Jane	20.0	3.0	Master	74444.444444	San Francisco	
2	Mike	40.0	5.0	PhD	60000.000000	Austin	
3	Anna	105.0	8.0	High School	65000.000000	Beijing	
4	Lucy	NaN	5.0	PhD	75000.000000	Berlin	
5	Emma	30.0	5.0	Master	55000.000000	Toronto	
6	Dave	45.0	20.0	PhD	120000.000000	London	
7	Sophia	38.0	10.0	Bachelor	65000.000000	Sydney	
8	Zhang	50.0	25.0	PhD	130000.000000	Tokyo	
9	Liam	27.0	4.0	Master	50000.000000	Dublin	

	department	languages_known	experience_category
0	Marketing	2	Junior
1	Engineering	3	Mid-level
2	Sales	1	Mid-level
3	Research	2	Senior
4	Design	3	Mid-level
5	HR	1	Mid-level
6	Finance	2	Senior
7	IT	3	Senior
8	Operations	2	Senior
9	Logistics	2	Mid-level

Question 9: (10 Points)

Identify and remove any duplicate rows from the dataframe.

```
In [80]: # Drop duplicate rows.
employees_df = employees_df.drop_duplicates()

print(employees_df)
```

	name	age	experience	Education_level	salary	location	\
0	John	25.0	2.0	Bachelor	50000.000000	New York	
1	Jane	20.0	3.0	Master	74444.444444	San Francisco	
2	Mike	40.0	5.0	PhD	60000.000000	Austin	
3	Anna	105.0	8.0	High School	65000.000000	Beijing	
4	Lucy	NaN	5.0	PhD	75000.000000	Berlin	
5	Emma	30.0	5.0	Master	55000.000000	Toronto	
6	Dave	45.0	20.0	PhD	120000.000000	London	
7	Sophia	38.0	10.0	Bachelor	65000.000000	Sydney	
8	Zhang	50.0	25.0	PhD	130000.000000	Tokyo	
9	Liam	27.0	4.0	Master	50000.000000	Dublin	

	department	languages_known	experience_category
0	Marketing	2	Junior
1	Engineering	3	Mid-level
2	Sales	1	Mid-level
3	Research	2	Senior
4	Design	3	Mid-level
5	HR	1	Mid-level
6	Finance	2	Senior
7	IT	3	Senior
8	Operations	2	Senior
9	Logistics	2	Mid-level

Question 10: (10 Points)

Find employees who have "PhD" in their education but a salary less than the mean salary of the dataframe.

```
In [81]: # Mean salary first.
mean_salary = employees_df["salary"].mean()

# PhD but making less than average.
low_paid_phd = employees_df[
    (employees_df["Education_level"] == "PhD") &
    (employees_df["salary"] < mean_salary)
]

print(low_paid_phd)
```

	name	age	experience	Education_level	salary	location	department	\
2	Mike	40.0	5.0	PhD	60000.0	Austin	Sales	
		languages_known	experience_category					
2		1	Mid-level					

Question 11: (10 Points)

Sort the dataframe based on the `experience` column in descending order.

```
In [82]: # Sort by experience.
sorted_employees = employees_df.sort_values(by="experience", ascending=False)

print(sorted_employees)
```

	name	age	experience	Education_level	salary	location	\
8	Zhang	50.0	25.0	PhD	130000.000000	Tokyo	
6	Dave	45.0	20.0	PhD	120000.000000	London	
7	Sophia	38.0	10.0	Bachelor	65000.000000	Sydney	
3	Anna	105.0	8.0	High School	65000.000000	Beijing	
5	Emma	30.0	5.0	Master	55000.000000	Toronto	
4	Lucy	NaN	5.0	PhD	75000.000000	Berlin	
2	Mike	40.0	5.0	PhD	60000.000000	Austin	
9	Liam	27.0	4.0	Master	50000.000000	Dublin	
1	Jane	20.0	3.0	Master	74444.444444	San Francisco	
0	John	25.0	2.0	Bachelor	50000.000000	New York	
	department	languages_known	experience_category				
8	Operations	2	Senior				
6	Finance	2	Senior				
7	IT	3	Senior				
3	Research	2	Senior				
5	HR	1	Mid-level				
4	Design	3	Mid-level				
2	Sales	1	Mid-level				
9	Logistics	2	Mid-level				
1	Engineering	3	Mid-level				
0	Marketing	2	Junior				

Question 12: (10 Points)

Group the dataframe by `education_level` and find the average salary for each education level.

```
In [83]: # Average salary by education.
avg_salary_by_education = employees_df.groupby("Education_level")["salary"].mean()
```

```
print(avg_salary_by_education)
```

```
Education_level
Bachelor      57500.000000
High School   65000.000000
Master        59814.814815
PhD           96250.000000
Name: salary, dtype: float64
```

Question 13: (10 Points)

Write a function that categorizes each employee's experience into 'Novice', 'Intermediate', or 'Expert' based on the number of years of experience:

- 'Novice' for less than 5 years.
- 'Intermediate' for 5 to 10 years.
- 'Expert' for more than 10 years.

Consider missing experience values as 'Unknown' and handle them appropriately in your function.

Apply this function to the experience column to create a new column named experience_level.

```
In [84]: # Experience labels again, but different.
def experience_type(x):
    if x < 5:
        return "Novice"
    elif x <= 10:
        return "Intermediate"
    else:
        return "Expert"

employees_df["experience_level"] = employees_df["experience"].apply(experience_type)

print(employees_df)
```

	name	age	experience	Education_level	salary	location	\
0	John	25.0	2.0	Bachelor	50000.000000	New York	
1	Jane	20.0	3.0	Master	74444.444444	San Francisco	
2	Mike	40.0	5.0	PhD	60000.000000	Austin	
3	Anna	105.0	8.0	High School	65000.000000	Beijing	
4	Lucy	NaN	5.0	PhD	75000.000000	Berlin	
5	Emma	30.0	5.0	Master	55000.000000	Toronto	
6	Dave	45.0	20.0	PhD	120000.000000	London	
7	Sophia	38.0	10.0	Bachelor	65000.000000	Sydney	
8	Zhang	50.0	25.0	PhD	130000.000000	Tokyo	
9	Liam	27.0	4.0	Master	50000.000000	Dublin	

	department	languages_known	experience_category	experience_level
0	Marketing	2	Junior	Novice
1	Engineering	3	Mid-level	Novice
2	Sales	1	Mid-level	Intermediate
3	Research	2	Senior	Intermediate
4	Design	3	Mid-level	Intermediate
5	HR	1	Mid-level	Intermediate
6	Finance	2	Senior	Expert
7	IT	3	Senior	Intermediate
8	Operations	2	Senior	Expert
9	Logistics	2	Mid-level	Novice

